

SMS Spam Classification

From Raw Text to a Deployed Flask App

NextBridge AI / Machine Learning Track — Summer Internship 2026

Task 3: NLP — Full Pipeline Walkthrough + Flask Deployment

Dataset: SMS Spam Collection (UCI / Kaggle) — 5,572 messages

Report generated from project run logs (preprocessing, features, classical models, GloVe embeddings, LSTM, and DistilBERT fine-tuning).

1. Project Overview

This report documents the end-to-end pipeline built for SMS spam classification: data inspection, text preprocessing, feature extraction (Bag-of-Words and TF-IDF), classical machine learning models, pre-trained word embeddings, a deep learning (LSTM) classifier, and a fine-tuned transformer (DistilBERT). Every approach is compared on the same held-out test split so that the trade-offs between simplicity, accuracy, and inference cost are directly visible.

The dataset is imbalanced (~87% ham / 13% spam), which shaped two decisions used throughout the project: the train/test split was stratified on the label, and evaluation was reported per-class (precision, recall, F1) rather than relying on overall accuracy, since a model that always predicts “ham” would already score close to 87% accuracy while being useless as a spam filter.

1.1 Dataset Summary

Property	Value
Total messages	5,572
Class balance	4,825 ham (86.6%) / 747 spam (13.4%)
Train / test split	4,457 train / 1,115 test (80/20, stratified)
Avg. message length	~80 characters / ~15 words
Longest message	171 words

2. Preprocessing Pipeline

Raw messages were cleaned with a single reusable **clean_text()** function so the exact same logic runs at training time and at inference time inside the Flask app:

- Lowercased all text
- Removed punctuation and numbers
- Tokenized with NLTK's **word_tokenize()** (handles punctuation edge cases better than a plain `.split()`)
- Removed English stopwords (`nltk.corpus.stopwords`)
- Lemmatized tokens with **WordNetLemmatizer** (kept as real words, e.g. “running” → “run”, rather than stemmed to non-words)

Example transformation observed in the cleaned dataset: “Go until jurong point, crazy.. Available only in bugis n great world...” → “go jurong point crazy available bugis n great...”.

3. Feature Extraction

Two vector representations were built strictly **after** the train/test split (fit only on the training set, then applied to test) to avoid vocabulary leakage:

Representation	Vocabulary size	Train shape	Test shape
TF-IDF (TfidfVectorizer)	5,000	(4457, 5000)	(1115, 5000)
Bag-of-Words (CountVectorizer)	5,000	(4457, 5000)	(1115, 5000)

Both vectorizers and all feature matrices were persisted with `joblib` under `models/` so that later stages (LSTM, comparison scripts) could reload them without recomputation.

4. Classical Models (TF-IDF Features)

Two classifiers were trained on the TF-IDF features: Multinomial Naive Bayes (the standard text-classification baseline) and a Linear SVM.

Model	Accuracy	Spam Precision	Spam Recall	Spam F1
TF-IDF + Multinomial Naive Bayes	0.9650	0.99	0.74	0.85
TF-IDF + Linear SVC	0.9883	0.986	0.926	0.955

Confusion matrices (test set, n=1,115):

- Naive Bayes — ham: 965 correct / 1 misclassified; spam: 111 correct / 38 missed as ham
- Linear SVC — ham: 964 correct / 2 misclassified; spam: 138 correct / 11 missed as ham

Observation: Naive Bayes is very precise on spam (0.99) but recall lags (0.74) — it misses roughly 1 in 4 spam messages. Linear SVC is both more precise and far more complete on recall (0.93), making it the stronger of the two classical models on this dataset.

5. Embedding Exploration (GloVe)

Pre-trained 100-dimensional GloVe vectors (glove.6B.100d, 400,000-word vocabulary) were averaged across each message's tokens to build one fixed-length vector per SMS, then fed into the same Multinomial Naive Bayes classifier used in Step 4.

Model	Accuracy	Spam Precision	Spam Recall	Spam F1
TF-IDF + Multinomial NB (baseline)	0.9650	0.99	0.74	0.85
GloVe (avg.) + Multinomial NB	0.8664	0.00	0.00	0.00

Observation: The GloVe + Naive Bayes model collapsed to predicting “ham” for every message (0% spam precision/recall), landing at 86.6% accuracy — almost exactly the no-information baseline for this class distribution. Averaged GloVe vectors contain negative values and a very different value range than TF-IDF counts; Multinomial NB assumes non-negative, count-like features, so it is a poor fit for averaged dense embeddings. A linear model (Logistic Regression / SVM) or a small MLP on top of the averaged GloVe vectors would be the correct comparison and would very likely score far higher than this result suggests — the low score here reflects a model/feature mismatch, not a limitation of GloVe embeddings themselves.

6. Deep Learning Classifier (LSTM)

Messages were tokenized with the Keras Tokenizer and padded/truncated to a fixed length of 50 tokens, then passed through a trainable embedding layer, an LSTM layer, and a sigmoid output layer, trained with binary cross-entropy and the Adam optimizer for up to 10 epochs.

Epoch	Train Acc.	Train Loss	Val Acc.	Val Loss
1	0.9187	0.2339	0.9652	0.1115
2	0.9874	0.0492	0.9742	0.0762
3	0.9955	0.0186	0.9709	0.0831
4	0.9978	0.0085	0.9720	0.0917
5	0.9986	0.0052	0.9563	0.1412

Model	Accuracy	Spam Precision	Spam Recall	Spam F1
LSTM	0.983	0.96	0.91	0.93

Overfitting evidence: validation loss reaches its minimum at epoch 2 (0.0762) while training loss keeps falling every epoch (down to 0.0052 by epoch 5). From epoch 3 onward, validation loss rises even as validation accuracy briefly holds and then drops (0.9742 → 0.9563), a textbook train/validation divergence. Training was allowed to run past the point an early-stopping callback would have halted it — epoch 2's weights, not epoch 5's, were the better checkpoint.

7. Transformer Fine-Tuning (DistilBERT)

distilbert-base-uncased was fine-tuned end-to-end with its built-in sequence classification head using the Hugging Face Trainer API (3 epochs, learning rate 2e-5, batch size 16, on a Colab T4 GPU) rather than being used only as a frozen feature extractor.

Model	Accuracy	Spam Precision	Spam Recall	Spam F1
DistilBERT (fine-tuned)	0.9901	0.9792	0.9463	0.9625

DistilBERT misclassified 11 of the 1,115 test messages (99.0% overall accuracy). Five representative failures:

True Label	Predicted	Message (cleaned)
spam	ham	latest news police station toilet stolen cop nothing go
ham	spam	luton 0125698789 ring ur around h
spam	ham	bought one ringtone getting text costing 3 pound offering tone etc
ham	spam	hey pple700 900 5 nightsexcellent location wif breakfast hamper
spam	ham	sale arsenal dartboard good condition double treble

8. Final Model Comparison

Model	Accuracy	Spam Precision	Spam Recall	Spam F1
TF-IDF + Multinomial NB	0.9650	0.99	0.74	0.85
TF-IDF + Linear SVC	0.9883	0.986	0.926	0.955
GloVe (avg.) + Multinomial NB	0.8664	0.00	0.00	0.00
LSTM	0.983	0.96	0.91	0.93
DistilBERT (fine-tuned)	0.9901	0.9792	0.9463	0.9625

Ranked by spam-class F1: **DistilBERT (0.963)** > TF-IDF + Linear SVC (0.955) > LSTM (0.93) > TF-IDF + Naive Bayes (0.85) > GloVe + Naive Bayes (0.00, mismatched setup — see Section 5).

9. Required Analysis Answers

Q1 — Best spam-class F1 among TF-IDF, LSTM, and BERT

DistilBERT scores highest (F1 = 0.963), narrowly ahead of TF-IDF + Linear SVC (0.955) and clearly ahead of LSTM (0.93) and TF-IDF + Naive Bayes (0.85). The gap between BERT and the best classical model (SVC) is small (~0.8 points of F1) — most of the available signal in this dataset is already captured by a linear model over TF-IDF features; the transformer's contextual understanding buys only a small additional improvement here.

Q2 — TF-IDF vs. plain Bag-of-Words

Bag-of-Words features were built (same 5,000-word vocabulary, same shapes as TF-IDF) but a model was trained and evaluated only on the TF-IDF version in the captured run — the BoW classifier's own accuracy/F1 numbers were not produced in the logs available for this report. To complete this comparison, train the same Multinomial NB / Linear SVC on `X_train_bow / X_test_bow` (already saved under `models/`) and add the resulting row to the comparison table. Conceptually, TF-IDF is expected to match or slightly outperform raw BoW here because it down-weights very common words (e.g. “call”, “free” appearing broadly) relative to words that are more distinctive of a specific message, which tends to sharpen the decision boundary for a linear/NB classifier on short texts like SMS.

Q3 — Did LSTM's word-order modeling actually help?

Not decisively. LSTM's test accuracy (0.983) and spam F1 (0.93) sit *below* the much simpler TF-IDF + Linear SVC (0.988 accuracy, 0.955 F1). The training curve (Section 6) shows clear overfitting: validation loss bottoms out at epoch 2 (0.076) and then rises through epoch 5 (0.141) while training loss keeps shrinking toward zero. With only 4,457 training messages, the LSTM had enough capacity to memorize the training set faster than it could generalize the value of word order, so on a dataset this size the extra architecture cost did not translate into a better spam filter than a well-tuned linear model.

Q4 — Top 15 words most associated with spam

This analysis was not run in the captured logs (the feature-importance step was not executed against the Linear SVC / Naive Bayes coefficients before this report was generated). To produce it: for Linear SVC, sort `svc.coef_[0]` against `tfidf.get_feature_names_out()` and take the 15 highest positive weights (spam-leaning); for Naive Bayes, take the 15 words with the largest difference between `feature_log_prob_[1]` (spam) and `feature_log_prob_[0]` (ham). From the misclassified/clean-text samples visible in this run, words like *free*, *win*, *call*, *text*, *claim*, *prize*, *urgent*, *mobile*, and *offering* are typical high-signal spam terms in this dataset and are a reasonable starting point to verify against the actual coefficients.

Q5 — Where does the best model fail?

Using DistilBERT (the best overall F1) as the reference, 11 of 1,115 test messages were misclassified (Section 7). The pattern in the five examples shown is consistent:

- **Spam** → **ham misses** tend to be short, informal-sounding messages that lack the classic spam vocabulary (“free”, “win”, “claim”) even though they describe a commercial transaction — e.g. a message about a stolen phone/ringtone billing, or a dartboard sale, reads more like a casual personal message than a bulk promotion.
- **Ham** → **spam false alarms** both contain long digit sequences or dense concatenated text (a phone number, a run-together “ppl700 900 5 nightsexcellent” style listing) — surface features (digits, unusual token density) that strongly correlate with spam elsewhere in the dataset, even though the message is a genuine personal one.

In short, the model's errors cluster around messages where the surface style (digits, promotional-sounding phrasing) disagrees with the true intent — exactly the boundary cases where even a human skimming the message quickly could plausibly misjudge it.

Q6 — Production recommendation

Recommended: TF-IDF + Linear SVC. It reaches 0.955 spam F1 — within 0.8 points of fine-tuned DistilBERT's 0.963 — while being orders of magnitude cheaper to run: inference is a sparse matrix-vector multiply (sub-millisecond on CPU, no GPU required), the whole pipeline (vectorizer + classifier) serializes to a few megabytes, and there is no tokenizer/model download or warm-up cost at deploy time. DistilBERT's small F1 edge does not clearly justify its cost: GPU-friendly inference, a much larger model artifact, and materially higher latency per request. LSTM is the weakest choice for production here — it is both less accurate than SVC and, per Q3, still overfits on a dataset this size, so it carries deep-learning operational cost without a corresponding accuracy benefit. DistilBERT would be worth revisiting if the message distribution grows more adversarial or context-dependent over time and the extra recall becomes business-critical; for this dataset and use case, Linear SVC is the pragmatic choice and is what the Flask app in Section 10 serves.

10. Deployment — Flask Web App

The best-performing practical model, **TF-IDF + Linear SVC** (see Q6), is packaged as a single `scikit-learn Pipeline` combining the fitted `TfidfVectorizer` and the trained `LinearSVC`, saved as `models/spam_pipeline.joblib`. Using one `Pipeline` object guarantees the vectorizer applied at inference time is exactly the one fit during training — avoiding the classic bug of a vocabulary mismatch between training and serving.

10.1 App structure

- `app.py` — Flask entry point; imports `clean_text()` directly from `src/preprocessing.py` so the exact training-time cleaning (lowercase, punctuation/number stripping, tokenize, stopwords removal, lemmatize) runs on every incoming message before prediction
- `templates/index.html` — a minimal form: a textarea for the message, a submit button, and a result area showing Spam/Ham plus a confidence percentage (from `decision_function / predict_proba`)
- Exception handling around model loading and prediction, including the empty-message case

10.2 Request flow

- User submits a message through the form
- Flask calls the shared `clean_text()` function
- The cleaned text is passed into `spam_pipeline.predict()` (vectorizes + classifies in one call)
- The predicted label (Spam/Ham) and a confidence score are rendered back to the page

Testing note: before considering deployment complete, the app was checked against messages outside the training set, including short edge cases and messages dense with digits/links, since those are exactly the categories where preprocessing mismatches most often surface (see Q5).

11. Strengths & Weaknesses by Approach

Approach	Strengths	Weaknesses
TF-IDF + Naive Bayes	Fast, tiny, very high spam precision	Lower recall — misses ~1 in 4 spam messages
TF-IDF + Linear SVC	Best accuracy/F1 trade-off for cost; cheap inference	Linear decision boundary; no context/word order
GloVe (avg.) + Naive Bayes	Reuses general-purpose pre-trained vectors	Poor fit for NB on this run — needs a linear/MLP head instead
LSTM	Learns word order and local context	Prone to overfitting at this dataset size; below SVC on F1
DistilBERT (fine-tuned)	Best raw F1; strong contextual understanding	Heaviest to deploy: GPU-friendly, larger artifact, higher latency

12. Repository & Reproduction Notes

- Environment: Python virtual environment; dependencies pinned in `pyproject.toml` / `requirements.txt`
- Run order: `preprocessing.py` → `features.py` → `train_and_save.py` → `embeddings.py` → `lstm_model.py` → BERT fine-tuning notebook → `app.py`
- `data/`, `models/`*, `joblib`, and other large artifacts are excluded from version control via `.gitignore`

Report compiled from the project's captured console/notebook output (preprocessing, feature extraction, classical model training, GloVe embeddings, LSTM training, and DistilBERT fine-tuning logs).